

LAPORAN PRAKTIKUM BASIS DATA NOSQL
AGGREGATION FRAMEWORK
PERTEMUAN 5 (TUTORIAL TERBIMBING, LANGKAH Pengerjaan,
LATIHAN MANDIRI)

225443053

SHIVA NAYLA ZAHRA

1AEC1

1AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2025/2026

BAB I

PENDAHULUAN

Pada tugas ini, mahasiswa diarahkan untuk memahami dan menerapkan pengolahan data menggunakan MongoDB melalui aggregation pipeline secara langsung tanpa bantuan aplikasi lain. Melalui praktikum ini, mahasiswa belajar membuat data simulasi yang menyerupai kondisi nyata di industri, serta menyusun pipeline bertahap dengan berbagai operator seperti '\$group', '\$match', '\$project', '\$lookup', '\$sort', '\$addFields', dan '\$facet'. Selain itu, mahasiswa juga dilatih untuk mengoptimalkan query, misalnya dengan menempatkan '\$match' di awal dan menggunakan indeks agar proses lebih efisien. Tidak hanya itu, hasil agregasi yang diperoleh juga dianalisis untuk membantu pengambilan keputusan, seperti menemukan mesin yang bermasalah, melihat tren cacat, serta menyusun rekomendasi perbaikan berdasarkan data yang ada.

BAB II

DESAIN DATA

2.1. Membuat database:

```
use stdk_mesin
```

Membuat 1000 dokumen untuk 10 mesin, periode 3 bulan (April–Juni 2026), 3 shift per hari:

```
var docs = [];
var mesinList = []; for (let i=1;i<=10;i++) mesinList.push("M"+i.toString().padStart(2,'0'));
var start = new Date(2026,3,1); // April
for (let i=0; i<1000; i++) {
  var tgl = new Date(start.getTime() + Math.floor(Math.random()*90)*86400000);
  var shift = Math.floor(Math.random()*3)+1;
  var target = Math.floor(Math.random()*301)+200;
  var actual_ok = Math.floor(target * (0.6 + Math.random()*0.4));
  var actual_reject = Math.floor(actual_ok * Math.random()*0.15);
  var durasi_tersedia = 480;
  var durasi_operasi = Math.floor(durasi_tersedia * (0.7 + Math.random()*0.3));
  docs.push({
    mesin: mesinList[Math.floor(Math.random()*10)],
    tanggal: tgl,
    shift: shift,
    target: target,
    actual_ok: actual_ok,
    actual_reject: actual_reject,
    durasi_operasi_menit: durasi_operasi,
    durasi_tersedia_menit: durasi_tersedia
  });
  if (docs.length === 500) { db.produksi_harian.insertMany(docs); docs = []; }
}
if (docs.length) db.produksi_harian.insertMany(docs);
```

Verifikasi data:

```
> db.produksi_harian.countDocuments()
< 1000
```

```
> db.produksi_harian.findOne()
< {
  _id: ObjectId('69f80f968ee63419b4c4d148'),
  mesin: 'M01',
  tanggal: 2026-04-15T17:00:00.000Z,
  shift: 3,
  target: 274,
  actual_ok: 254,
  actual_reject: 32,
  durasi_operasi_menit: 448,
  durasi_tersedia_menit: 480
}
```

BAB III

TUTORIAL TERBIMBING

3.1. Pipeline Lengkap:

```
db.produksi_harian.aggregate([
// Tahap 1: Tidak perlu $match karena ingin semua data
// Tahap 2: Group by mesin dan bulan
{ $group: {
  _id: {
    mesin: "$mesin",
    bulan: { $month: "$tanggal" },
    tahun: { $year: "$tanggal" }
  },
  total_target: { $sum: "$target" },
  total_ok: { $sum: "$actual_ok" },
  total_reject: { $sum: "$actual_reject" },
  total_op: { $sum: "$durasi_operasi_menit" },
  total_avail: { $sum: "$durasi_tersedia_menit" }
}},
// Tahap 3: Hitung OEE
{ $project: {
  _id: 0,
  mesin: "$_id.mesin",
  bulan: "$_id.bulan",
  tahun: "$_id.tahun",
  total_ok: 1,
  total_target: 1,
  total_reject: 1,
  availability: { $divide: ["$total_op", "$total_avail"] },
  performance: { $divide: ["$total_ok", "$total_target"] },
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
  OEE: {
    $multiply: [
      { $divide: ["$total_ok", "$total_target"] },
      { $divide: ["$total_op", "$total_avail"] },
      { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
    ]
  }
}},
// Tahap 4: Urutkan berdasarkan OEE terendah
{ $sort: { OEE: 1 } }
])
```

Screenshoot Output:

>_MONGOSH	>_MONGOSH
<pre> < { total_target: 251, total_ok: 167, total_reject: 8, mesin: 'M02', bulan: 3, tahun: 2026, availability: 0.7288333333333333, performance: 0.6653386454183267, quality: 0.9542857142857143, OEE: 0.4576737810662113 } { total_target: 422, total_ok: 301, total_reject: 8, mesin: 'M10', bulan: 3, tahun: 2026, availability: 0.70625, performance: 0.7132701421800948, quality: 0.9741100323624595, OEE: 0.49970564348557375 } { total_target: 782, total_ok: 576, total_reject: 31, mesin: 'M01', bulan: 3, tahun: 2026, availability: 0.7125, performance: 0.7365728900255755, quality: 0.9489291598023064, OEE: 0.4980057892364022 } </pre>	<pre> { total_target: 803, total_ok: 710, total_reject: 96, mesin: 'M07', bulan: 3, tahun: 2026, availability: 0.7229166666666667, performance: 0.8841843088418431, quality: 0.8808933002481389, OEE: 0.5630595744658935 } { total_target: 687, total_ok: 580, total_reject: 17, mesin: 'M04', bulan: 3, tahun: 2026, availability: 0.809375, performance: 0.727802037845706, quality: 0.9671179883945842, OEE: 0.5696951396338185 } { total_target: 873, total_ok: 695, total_reject: 34, mesin: 'M05', bulan: 3, tahun: 2026, availability: 0.7638888888888888, performance: 0.7961053837342497, quality: 0.953360768175583, OEE: 0.5797730584751123 } </pre>

>_MONGOSH	>_MONGOSH
<pre> { total_target: 13774, total_ok: 10465, total_reject: 721, mesin: 'M07', bulan: 4, tahun: 2026, availability: 0.8326841666666667, performance: 0.759764774212284, quality: 0.9355444305381727, OEE: 0.5918097987860338 } { total_target: 10694, total_ok: 8185, total_reject: 593, mesin: 'M03', bulan: 5, tahun: 2026, availability: 0.8520138888888888, performance: 0.7579016270806059, quality: 0.9318234076799264, OEE: 0.6017181750178369 } { total_target: 11368, total_ok: 8810, total_reject: 604, mesin: 'M02', bulan: 4, tahun: 2026, availability: 0.8300130208333333, performance: 0.7749824867558058, quality: 0.9358402379434885, OEE: 0.6019750110364444 } </pre>	<pre> { total_target: 8775, total_ok: 6658, total_reject: 453, mesin: 'M03', bulan: 6, tahun: 2026, availability: 0.8477564102564102, performance: 0.7587464387464388, quality: 0.9362958796231191, OEE: 0.6022556184335515 } { total_target: 11201, total_ok: 8689, total_reject: 645, mesin: 'M02', bulan: 5, tahun: 2026, availability: 0.8377380952380953, performance: 0.7757343094366574, quality: 0.9308977930147847, OEE: 0.6049552717307124 } { total_target: 12876, total_ok: 10086, total_reject: 877, mesin: 'M01', bulan: 4, tahun: 2026, availability: 0.8460069444444445, performance: 0.7833178005591799, quality: 0.9200036486363222, OEE: 0.6096793329848058 } </pre>

Penjelasan:

Pada bagian ini, pipeline digunakan untuk mengagregasi seluruh data produksi dan menghitung nilai OEE melalui beberapa tahap utama. Pertama, `\$group` mengelompokkan dokumen berdasarkan mesin dan bulan. Selanjutnya, `\$project` membentuk output sekaligus menghitung komponen OEE menggunakan `\$divide` dan `\$multiply`. Terakhir, `\$sort` digunakan untuk mengurutkan nilai OEE dari yang terendah (ascending). Membuat 1000 dokumen untuk 10 mesin, periode 3 bulan (April–Juni 2026), 3 shift per hari.

3.2. Menemukan mesin dengan OEE di bawah 80%:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: {
      mesin: "$mesin",
      bulan: { $month: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    total_target: { $sum: "$target" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_op: { $sum: "$durasi_operasi_menit" },
    total_avail: { $sum: "$durasi_tersedia_menit" }
  }
},
//Hitung OEE
{ $project: {
  _id: 0,
  mesin: "$_id.mesin",
  bulan: "$_id.bulan",
  tahun: "$_id.tahun",
  availability: { $divide: ["$total_op", "$total_avail"] },
  performance: { $divide: ["$total_ok", "$total_target"] },
  quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
  OEE: {
    $multiply: [
      { $divide: ["$total_ok", "$total_target"] },
      { $divide: ["$total_op", "$total_avail"] },
      { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] }
    ]
  }
}
},
//Filter mesin dengan OEE di bawah 80% (0.80)
{ $match: {
  OEE: { $lt: 0.80 }
}
},
//Urutkan dari OEE yang paling rendah
{ $sort: { OEE: 1 }
}
])
```

Screenshoot Output:

>_MONGOOSH	>_MONGOOSH
<pre>< { mesin: 'M02', bulan: 3, tahun: 2026, availability: 0.7208333333333333, performance: 0.6653386454183267, quality: 0.9542857142857143, OEE: 0.4576737810662113 } { mesin: 'M10', bulan: 3, tahun: 2026, availability: 0.70625, performance: 0.7132701421800948, quality: 0.9741100323624595, OEE: 0.49078504340557375 } { mesin: 'M01', bulan: 3, tahun: 2026, availability: 0.7125, performance: 0.7365728900255755, quality: 0.9489291598023064, OEE: 0.4980057892364022 } { mesin: 'M07', bulan: 3, tahun: 2026, availability: 0.7220166666666667, performance: 0.8841843088418431, quality: 0.8808933002481389, OEE: 0.5630595744658935 } }</pre>	<pre>{ mesin: 'M04', bulan: 3, tahun: 2026, availability: 0.809375, performance: 0.727802037845706, quality: 0.9671179883945842, OEE: 0.5696951396338185 } { mesin: 'M05', bulan: 3, tahun: 2026, availability: 0.7638888888888888, performance: 0.7961053837342497, quality: 0.953360768175583, OEE: 0.5797730584751123 } { mesin: 'M07', bulan: 4, tahun: 2026, availability: 0.8326041666666667, performance: 0.759764774212284, quality: 0.9355444305381727, OEE: 0.5918097987860338 } { mesin: 'M03', bulan: 5, tahun: 2026, availability: 0.8520138888888888, performance: 0.7579016270806059, quality: 0.9318234076799264, OEE: 0.6017181750178369 } }</pre>

>_MONGOOSH
<pre>{ mesin: 'M02', bulan: 4, tahun: 2026, availability: 0.8300130208333333, performance: 0.7749824067558058, quality: 0.9358402379434885, OEE: 0.6019750110364444 } { mesin: 'M03', bulan: 6, tahun: 2026, availability: 0.8477564102564102, performance: 0.7587464387464388, quality: 0.9362958796231191, OEE: 0.6022556184335515 } { mesin: 'M02', bulan: 5, tahun: 2026, availability: 0.8377380952380953, performance: 0.7757343894366574, quality: 0.9308977930147847, OEE: 0.6049552717307124 } { mesin: 'M01', bulan: 4, tahun: 2026, availability: 0.8460069444444445, performance: 0.7833178005591799, quality: 0.9200036486363222, OEE: 0.6096793329848058 } }</pre>

Penjelasan: Pada bagian ini, pipeline digunakan untuk mengolah seluruh data produksi dan menghitung nilai OEE melalui beberapa tahap utama. Pertama, `\$group` mengelompokkan dokumen berdasarkan mesin dan bulan. Selanjutnya, `\$project` membentuk output sekaligus menghitung komponen OEE dengan `\$divide` dan `\$multiply`. Setelah itu, `\$match` dengan operator `\$lt` digunakan untuk menyaring data dengan nilai OEE di bawah 80%. Terakhir, `\$sort` mengurutkan hasil berdasarkan nilai OEE dari yang terendah (ascending).

3.3. Distribusi jumlah produksi per shift dalam bucket:

```
db.produksi_harian.aggregate([
  { $bucket: {
    groupBy: "$actual_ok",
    boundaries: [0, 200, 300, 400, 600], // 0-199, 200-299, 300-399, 400-600
    default: "di atas 600",
    output: {
      count: { $sum: 1 },
      rata_reject: { $avg: "$actual_reject" }
    }
  }
}]
```

Screenshoot Ouput:

```
< {
  _id: 0,
  count: 199,
  rata_reject: 12.984924623115578
}
{
  _id: 200,
  count: 428,
  rata_reject: 17.957943925233646
}
{
  _id: 300,
  count: 288,
  rata_reject: 26.375
}
{
  _id: 400,
  count: 85,
  rata_reject: 28.988235294117647
}
```

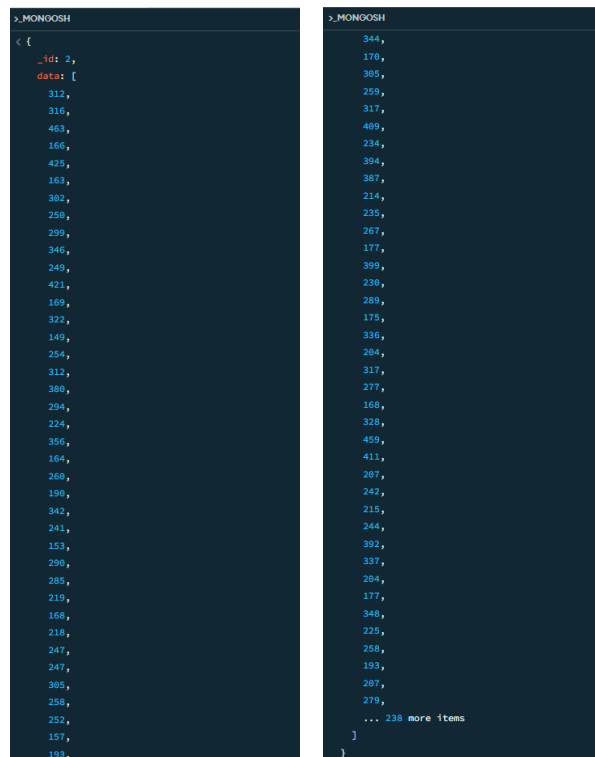
Penjelasan: Pada bagian ini, pipeline digunakan untuk mengolah data produksi dan menyusun distribusi nilai OEE melalui beberapa tahap utama. Pertama, `\$bucket` digunakan untuk mengelompokkan nilai actual_ok ke dalam rentang tertentu (misalnya 200–299, 300–399, dan seterusnya). Setiap bucket kemudian menampilkan jumlah dokumen serta rata-rata reject.

Namun, pada kode tersebut distribusi masih belum dipisahkan berdasarkan shift, sehingga diperlukan penyesuaian tambahan untuk menampilkan distribusi sesuai masing-masing shift.

3.4. Distribusi per shift, pakai \$group:

```
db.produksi_harian.aggregate([
  { $group: {
    _id: "$shift",
    data: { $push: "$actual_ok" } // kumpulkan nilai
  }},
])
```

Screenshot Output:



```
>_MONOOSH
< {
  _id: 2,
  data: [
    312,
    316,
    463,
    166,
    425,
    163,
    382,
    258,
    299,
    346,
    249,
    421,
    169,
    322,
    149,
    254,
    312,
    388,
    294,
    224,
    356,
    164,
    268,
    198,
    342,
    241,
    153,
    298,
    285,
    219,
    168,
    218,
    247,
    247,
    385,
    258,
    252,
    157,
    193,
    344,
    178,
    385,
    259,
    317,
    489,
    234,
    394,
    387,
    214,
    235,
    267,
    177,
    399,
    238,
    289,
    175,
    336,
    284,
    317,
    277,
    168,
    328,
    459,
    411,
    287,
    242,
    215,
    244,
    392,
    337,
    284,
    177,
    348,
    225,
    258,
    199,
    287,
    279,
    ... 238 more items
  ]
}
```

```
>_MONGOOSH
{
  "_id": 3,
  "data": [
    254,
    269,
    335,
    346,
    323,
    277,
    277,
    198,
    295,
    215,
    224,
    369,
    228,
    211,
    242,
    156,
    198,
    177,
    311,
    159,
    325,
    346,
    217,
    418,
    233,
    200,
    199,
    159,
    223,
    214,
    234,
    176,
    151,
    248,
    201,
    424,
    311,
    189,
    211,
    303,
    331,
    403,
    206,
    155,
    285,
    143,
    268,
    266,
    243,
    400,
    139,
    236,
    199,
    267,
    213,
    135,
    165,
    345,
    320,
    350,
    189,
    289,
    427,
    383,
    331,
    167,
    436,
    311,
    371,
    147,
    418,
    312,
    205,
    431,
    451,
    158,
    199,
    440,
    ... 238 more items
  ]
}
```

```
>_MONGOOSH
{
  "_id": 1,
  "data": [
    423,
    415,
    287,
    327,
    348,
    263,
    181,
    374,
    212,
    276,
    277,
    187,
    241,
    278,
    335,
    227,
    278,
    333,
    318,
    258,
    165,
    402,
    290,
    217,
    171,
    365,
    276,
    288,
    181,
    285,
    177,
    210,
    238,
    226,
    220,
    256,
    253,
    362,
    332,
    175,
    237,
    393,
    395,
    418,
    189,
    220,
    373,
    161,
    224,
    211,
    399,
    327,
    366,
    276,
    304,
    292,
    164,
    215,
    286,
    243,
    297,
    133,
    240,
    300,
    253,
    191,
    219,
    343,
    346,
    313,
    299,
    293,
    341,
    298,
    255,
    164,
    410,
    ... 224 more items
  ]
}
```

Penjelasan: Pada bagian ini, pipeline mengagregasi seluruh data produksi dan menghitung nilai OEE, yang terdiri dari beberapa stage utama. Pertama, \$group digunakan untuk mengelompokkan data berdasarkan shift. Selanjutnya, \$push digunakan untuk menambahkan

nilai actual_ok dan mengoutput tiap bucket berisi jumlah dokumen dan rata_reject. Untuk distribusi per shift, bisa pakai \$group.

3.5. Alternatif gunakan \$facet untuk menjalankan pipeline bucket paralel per shift:

```
db.produksi_harian.aggregate([
  { $facet: {
    "shift1": [
      { $match: { shift: 1 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: { count:
        {$sum:1} } } }
    ],
    "shift2": [
      { $match: { shift: 2 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: { count:
        {$sum:1} } } }
    ],
    "shift3": [
      { $match: { shift: 3 } },
      { $bucket: { groupBy: "$actual_ok", boundaries: [0,200,300,400,600], default: "600+", output: { count:
        {$sum:1} } } }
    ]
  } }
])
```

Screenshoot Output:

```

>_MONGOSH
< {
  shift1: [
    {
      _id: 0,
      count: 53
    },
    {
      _id: 200,
      count: 153
    },
    {
      _id: 300,
      count: 89
    },
    {
      _id: 400,
      count: 29
    }
  ],
  shift2: [
    {
      _id: 0,
      count: 75
    },
    {
      _id: 200,
      count: 135
    },
    {
      _id: 300,
      count: 102
    },
    {
      _id: 400,
      count: 26
    }
  ],
  shift3: [
    {
      _id: 0,
      count: 71
    },
    {
      _id: 200,
      count: 140
    },
    {
      _id: 300,
      count: 97
    },
    {
      _id: 400,
      count: 30
    }
  ]
}

```

Penjelasan: Pada bagian ini, pipeline digunakan untuk mengolah data produksi melalui beberapa tahap utama. Pertama, `\$facet` digunakan untuk menjalankan beberapa pipeline secara bersamaan pada data yang sama. Selanjutnya, `\$match` memfilter data berdasarkan shift tertentu. Kemudian, `\$bucket` mengelompokkan nilai actual_ok ke dalam rentang yang telah ditentukan (misalnya 200–299, 300–399, dan seterusnya). Hasil setiap bucket menampilkan jumlah data (count) yang dihitung menggunakan `\$sum`.

3.6. Cara lain lebih sederhana: Jika hanya ingin jumlah per shift, gunakan \$group langsung:

```

db.produksi_harian.aggregate([
  { $group: { _id: "$shift", total_produksi: { $sum: "$actual_ok" } } }
])

```

```
< {
  _id: 2,
  total_produksi: 92558
}
{
  _id: 3,
  total_produksi: 92663
}
{
  _id: 1,
  total_produksi: 90969
}
```

Penjelasan: Pada bagian ini, pipeline mengagregasi seluruh data produksi dan menghitung nilai OEE, yang terdiri dari beberapa stage utama. Pertama, \$group digunakan untuk mengelompokkan data sesuai shift, lalu total produksi dijumlahkan menggunakan \$sum.

3.7. Indeks: Buat indeks compound untuk tanggal dan mesin agar query OEE lebih cepat

```
db.produksi_harian.createIndex({ mesin: 1, tanggal: 1 })
```

Screenshoot Output

```
> db.produksi_harian.createIndex({ mesin: 1, tanggal: 1 })
< mesin_1_tanggal_1
```

Penjelasan: Pada bagian ini, syntax digunakan untuk membuat index dengan nilai mesin: 1, tanggal: 1, selanjutnya akan membuat index sehingga memudahkan pengguna untuk mencari data.

3.8. Latihan Tambahan

```
db.produksi_harian.aggregate([
{
  $group: {
    _id: { $dateToString: { format: "%Y-%m", date: "$tanggal" } },
    total_operasi: { $sum: "$durasi_operasi_menit" },
    total_tersedia: { $sum: "$durasi_tersedia_menit" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_target: { $sum: "$target" }
  }
},
{
  $project: {
    availability: { $divide: ["$total_operasi", "$total_tersedia"] },
    performance: { $divide: [{ $add: ["$total_ok", "$total_reject"] }, "$total_target"] },
    quality: { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] } ] },
    oee: {
      $multiply: [
        { $divide: ["$total_operasi", "$total_tersedia"] },
```

```

    { $divide: [{ $add: ["$total_ok", "$total_reject"] }, "$total_target"] },
    { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] }] }
  ]
}
},
{
  $sort: { _id: 1 }
}
])

```

Screenshoot Output

```

< {
  _id: '2026-03',
  availability: 0.7674679487179488,
  performance: 0.8419869633625534,
  quality: 0.9332621462893753,
  oee: 0.603072139518532
}
{
  _id: '2026-04',
  availability: 0.8470931661750246,
  performance: 0.8575207577085159,
  quality: 0.9301680598670702,
  oee: 0.6756740542315661
}
{
  _id: '2026-05',
  availability: 0.8450306372549019,
  performance: 0.8533208287151505,
  quality: 0.9304098475311232,
  oee: 0.6709020203923073
}
{
  _id: '2026-06',
  availability: 0.856939935064935,
  performance: 0.8418160542944068,
  quality: 0.9339575433407097,
  oee: 0.6737437048091155
}

```

Penjelasan: Modifikasi pipeline agar menampilkan OEE per bulan untuk semua mesin, lalu mengurutkan bulan.

```

db.produksi_harian.aggregate([
{
  $match: {
    tanggal: {
      $gte: ISODate("2026-06-01T00:00:00Z"),
      $lt: ISODate("2026-07-01T00:00:00Z")
    }
  }
}
])

```

```

    }
  },
  {
    $group: {
      _id: "$mesin",
      total_operasi: { $sum: "$durasi_operasi_menit" },
      total_tersedia: { $sum: "$durasi_tersedia_menit" },
      total_ok: { $sum: "$actual_ok" },
      total_reject: { $sum: "$actual_reject" },
      total_target: { $sum: "$target" }
    }
  },
  {
    $project: {
      oee: {
        $multiply: [
          { $divide: ["$total_operasi", "$total_tersedia"] },
          { $divide: [{ $add: ["$total_ok", "$total_reject"] }, "$total_target"] },
          { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] }] }
        ]
      }
    }
  },
  {
    $sort: { oee: 1 }
  },
  {
    $limit: 3
  }
]

```

Screenshoot Output

```

< {
  _id: 'M03',
  oee: 0.6432321572065162
}
{
  _id: 'M10',
  oee: 0.6555419341537169
}
{
  _id: 'M04',
  oee: 0.6708981460899962
}

```

Penjelasan: Menampilkan 3 mesin dengan OEE terendah di bulan Juni 2026 saja (menggunakan \$match di awal untuk filter bulan).

```
db.produksi_harian.aggregate([
{
  $group: {
    _id: {
      mesin: "$mesin",
      tanggal: { $dateToString: { format: "%Y-%m-%d", date: "$tanggal" } }
    },
    total_operasi: { $sum: "$durasi_operasi_menit" },
    total_tersedia: { $sum: "$durasi_tersedia_menit" },
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" },
    total_target: { $sum: "$target" }
  },
},
{
  $project: {
    oee: {
      $multiply: [
        { $divide: ["$total_operasi", "$total_tersedia"] },
        { $divide: [{ $add: ["$total_ok", "$total_reject"] }, "$total_target"] },
        { $divide: ["$total_ok", { $add: ["$total_ok", "$total_reject"] }] }
      ]
    }
  }
},
{
  $sort: { "_id.tanggal": 1, "_id.mesin": 1 }
}
])
```

Screenshoot Output


```

>_MONGOSH
< {
  _id: {
    mesin: 'M01',
    tanggal: '2026-03-31'
  },
  oee: 0.5248081841432225
}
{
  _id: {
    mesin: 'M02',
    tanggal: '2026-03-31'
  },
  oee: 0.4795982735723772
}
{
  _id: {
    mesin: 'M04',
    tanggal: '2026-03-31'
  },
  oee: 0.5890647743813683
}
{
  _id: {
    mesin: 'M05',
    tanggal: '2026-03-31'
  },
  oee: 0.6081360570192185
}
{
  _id: {
    mesin: 'M07',
    tanggal: '2026-03-31'
  },
  oee: 0.6391915732669158
}
}

```

Penjelasan: Menghitung OEE per mesin per hari (gunakan \$dateToString untuk ekstrak tanggal).

```

db.produksi_harian.aggregate([
{
  $group: {
    _id: "$mesin",
    total_ok: { $sum: "$actual_ok" },
    total_reject: { $sum: "$actual_reject" }
  }
},
{
  $project: {
    quality: {
      $divide: [
        "$total_ok",
        { $add: ["$total_ok", "$total_reject"] }
      ]
    }
  }
},
{
  $sort: {
    quality: -1
  }
}
])

```

```
{
  $match: {
    quality: { $lt: 0.95 }
  },
  {
    $sort: { quality: 1 }
  }
}]
```

Screenshoot Output

```
>_MONGOSH
< {
  _id: 'M05',
  quality: 0.927743804312842
}
{
  _id: 'M01',
  quality: 0.9278872118473039
}
{
  _id: 'M06',
  quality: 0.9297256925583922
}
{
  _id: 'M08',
  quality: 0.9305466854004343
}
{
  _id: 'M09',
  quality: 0.9314226835597619
}
{
  _id: 'M02',
  quality: 0.9318110291289297
}
{
  _id: 'M03',
  quality: 0.9323462414578587
}
{
  _id: 'M10',
  quality: 0.9335096588436171
}
{
  _id: 'M07',
  quality: 0.9337131100727082
}
```

Penjelasan: Simulasi perbaikan, jika quality harus minimal 95%, menampilkan mesin yang tidak memenuhi kriteria tersebut.

BAB IV

LANGKAH Pengerjaan

4.1. Membuat Database:

```
use tugas5_225443053
```

4.2. Generate data dummy (minimal 1000 dokumen untuk inspeksi, 50 untuk target_kualitas):

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

# Ganti dengan NIM Anda
NIM = "225443053"
client = MongoClient('mongodb://localhost:27017')
db = client[f'tugas5_{NIM}']
db.inspeksi.drop()
db.target_kualitas.drop()

mesin_list = [f"M{i:02d}" for i in range(1, 11)]
batch_ids = [f"B{str(i).zfill(4)}" for i in range(1, 2001)]
inspektor_list = ["Andi", "Budi", "Citra", "Dian", "Eka"]
jenis_cacat_options = ["gores", "retak", "bengkok", "warna", "dimensi", "pori"]

# Generate data inspeksi (1500 dokumen)
docs = []
start_date = datetime(2026, 4, 1, 6, 0)
for i in range(1500):
    batch = random.choice(batch_ids)
    mesin = random.choice(mesin_list)
    tanggal = start_date + timedelta(minutes=random.randint(0, 60*24*90)) # 3
    bulan
    shift = 1 if tanggal.hour < 14 else (2 if tanggal.hour < 22 else 3)
    jumlah = random.randint(100, 500)
    reject = int(jumlah * random.uniform(0, 0.10)) # 0-10% cacat
    jenis = random.sample(jenis_cacat_options, k=random.randint(1, 3))
    doc = {
        "batch_id": batch,
        "mesin": mesin,
        "tanggal": tanggal,
        "shift": shift,
        "inspektor": random.choice(inspektor_list),
        "jumlah_diperiksa": jumlah,
        "cacat_ditemukan": reject,
        "jenis_cacat": jenis
    }
```

```

docs.append(doc)
if len(docs) >= 500:
    db.inspeksi.insert_many(docs)
    docs.clear()
if docs:
    db.inspeksi.insert_many(docs)

print("Data inspeksi selesai. Total:", db.inspeksi.count_documents({}))

# Generate target kualitas (setiap batch punya target cacat maksimal 5% dari
rata-rata ukuran batch)
targets = []
for i in range(1, 2001):
    batch = f"B{str(i).zfill(4)}"
    # target cacat acak 1-15
    target_cacat = random.randint(1, 15)
    targets.append({"batch_id": batch, "target_maks_cacat": target_cacat})
    if len(targets) >= 500:
        db.target_kualitas.insert_many(targets)
        targets.clear()
if targets:
    db.target_kualitas.insert_many(targets)

print("Data target kualitas selesai. Total:",
db.target_kualitas.count_documents({}))

```

Screenshot Output:

```

Data inspeksi selesai. Total: 1500
Data target kualitas selesai. Total: 2000

```

4.3. Memastikan data siap:

```

use tugas5_225443053

db.inspeksi.countDocuments()

db.target_kualitas.countDocuments()

db.inspeksi.findOne()

db.target_kualitas.findOne()

```

Screenshot Output:

```

>_MONGOSH

> use tugas5_225443053
< switched to db tugas5_225443053
> db.inspeksi.countDocuments()
< 1500
> db.target_kualitas.countDocuments()
< 2000
> db.inspeksi.findOne()
< {
  _id: ObjectId('69f84b0315a116e78eed8749'),
  batch_id: 'B1957',
  mesin: 'M04',
  tanggal: 2026-06-27T13:55:00.000Z,
  shift: 1,
  inspektor: 'Citra',
  jumlah_diperiksa: 495,
  cacat_ditemukan: 21,
  jenis_cacat: [
    'retak',
    'gores'
  ]
}
> db.target_kualitas.findOne()
< {
  _id: ObjectId('69f84b0315a116e78eed8d25'),
  batch_id: 'B0001',
  target_maks_cacat: 10
}

```

4.4. Indeks untuk mempercepat agregasi:

```

db.inspeksi.createIndex({ batch_id: 1 })

db.inspeksi.createIndex({ mesin: 1, tanggal: 1 })

db.target_kualitas.createIndex({ batch_id: 1 })

```

4.5. Pipeline 1, Persentase cacat per batch:

```

{
  $addFields: {
    persen_cacat: {
      $multiply: [
        { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
        100
      ]
    }
  }
},
{ $sort: { persen_cacat: -1 } },
{ $limit: 10 }

```

Screenshoot Output

```

>_MONGODB
< {
  _id: ObjectId('69f84b0315a116e78eed8a36'),
  batch_id: 'B1086',
  mesin: 'M04',
  tanggal: 2026-06-27T22:47:00.000Z,
  shift: 3,
  inspektor: 'Citra',
  jumlah_diperiksa: 432,
  cacat_ditemukan: 43,
  jenis_cacat: [
    'dimensi',
    'pori',
    'warna'
  ],
  persen_cacat: 9.953703703703704
}
{
  _id: ObjectId('69f84b0315a116e78eed8b3c'),
  batch_id: 'B1346',
  mesin: 'M04',
  tanggal: 2026-06-18T11:05:00.000Z,
  shift: 1,
  inspektor: 'Andi',
  jumlah_diperiksa: 131,
  cacat_ditemukan: 13,
  jenis_cacat: [
    'dimensi'
  ],
  persen_cacat: 9.923664122137405
}
{
  _id: ObjectId('69f84b0315a116e78eed87dd'),
  batch_id: 'B0887',
  mesin: 'M07',
  tanggal: 2026-05-03T19:28:00.000Z,
  shift: 2,
  inspektor: 'Eka',
  jumlah_diperiksa: 415,
  cacat_ditemukan: 41,
  jenis_cacat: [
    'pori'
  ],
  persen_cacat: 9.879518072289157
}
{
  _id: ObjectId('69f84b0315a116e78eed8872'),
  batch_id: 'B1496',
  mesin: 'M06',
  tanggal: 2026-04-21T04:53:00.000Z,
  shift: 1,
  inspektor: 'Eka',

```

Penjelasan: Tahap pertama menggunakan \$addField untuk menambahkan field baru bernama persen_cacat yang dihitung dengan operasi matematika melalui \$multiply. Selanjutnya, \$sort digunakan untuk mengurutkan data dari nilai persentase cacat tertinggi. Terakhir, \$limit berfungsi untuk membatasi hasil sehingga hanya menampilkan 10 data teratas.

4.6. Pipeline 2, Gabung dengan Target dan Tentukan Status Batch:

```

db.inspeksi.aggregate([
  {
    $lookup: {
      from: "target_kualitas",
      localField: "batch_id",

```

```

    foreignField: "batch_id",
    as: "target_info"
  }
},
{
  $unwind: {
    path: "$target_info",
    preserveNullAndEmptyArrays: true
  }
},
{
  $addFields: {
    persen_cacat: {
      $cond: {
        if: { $eq: ["$jumlah_diperiksa", 0] },
        then: 0,
        else: {
          $multiply: [
            { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] },
            100
          ]
        }
      }
    },
    target_maks: "$target_info.target_maks_cacat"
  }
},
{
  $addFields: {
    status: {
      $cond: {
        if: { $gt: ["$persen_cacat", "$target_maks"] },
        then: "NOT OK",
        else: "OK"
      }
    }
  }
},
{
  $project: {
    _id: 0,
    batch_id: 1,
    mesin: 1,
    persen_cacat: 1,
    target_maks: 1,
    status: 1,
    selisih: {
      $subtract: [
        "$persen_cacat",
        { $ifNull: ["$target_maks", 0] }
      ]
    }
  }
},
{ $sort: { selisih: -1 } },

```

```
{ $limit: 10 }  
])
```

Screenshoot Output:

```
>_MONGOSH  
< {  
  batch_id: 'B0307',  
  mesin: 'M04',  
  persen_cacat: 9.602649006622517,  
  target_maks: 1,  
  status: 'NOT OK',  
  selisih: 8.602649006622517  
}  
{  
  batch_id: 'B0248',  
  mesin: 'M10',  
  persen_cacat: 9.574468085106384,  
  target_maks: 1,  
  status: 'NOT OK',  
  selisih: 8.574468085106384  
}  
{  
  batch_id: 'B0036',  
  mesin: 'M09',  
  persen_cacat: 9.475806451612904,  
  target_maks: 1,  
  status: 'NOT OK',  
  selisih: 8.475806451612904  
}  
{  
  batch_id: 'B0483',  
  mesin: 'M05',  
  persen_cacat: 9.375,  
  target_maks: 1,  
  status: 'NOT OK',  
  selisih: 8.375  
}  
{  
  batch_id: 'B1636',  
  mesin: 'M06',  
  persen_cacat: 9.297052154195011,  
  target_maks: 1,  
  status: 'NOT OK',  
  selisih: 8.297052154195011  
}
```

Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama. Pertama, `\$lookup` dan `\$unwind` digunakan untuk menggabungkan serta menampilkan data standar dari koleksi lain. Selanjutnya, `\$addFields` dipakai untuk menghitung *persen_cacat* sekaligus menentukan status berdasarkan target maksimum. Setelah itu, `\$project` digunakan untuk menghitung selisih antara nilai aktual dan target, lalu `\$sort` mengurutkan data dari selisih terbesar. Terakhir, `\$limit` membatasi hasil agar hanya menampilkan 10 data teratas.

4.7. Pipeline 3, Jumlah Batch NOT OK per Mesin per Minggu:

```
db.inspeksi.aggregate([
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: [ "$cacat_ditemukan", "$jumlah_diperiksa" ] }, 100 ] }
  }},
  { $addFields: {
    status: { $cond: { if: { $gt: [ "$persen_cacat", "$target_info.target_maks_cacat" ] }, then: "NOT OK", else: "OK" } }
  }},
  { $match: { status: "NOT OK" } },
  { $group: {
    _id: {
      mesin: "$mesin",
      minggu: { $week: "$tanggal" },
      tahun: { $year: "$tanggal" }
    },
    jumlah_NOT_OK: { $sum: 1 }
  }},
  { $sort: { "_id.tahun": 1, "_id.minggu": 1, "_id.mesin": 1 } }
])
```

Screenshoot Output

```

>_MONGOSH
< {
  _id: {
    mesin: 'M01',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 4
}
{
  _id: {
    mesin: 'M02',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M03',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M05',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}
{
  _id: {
    mesin: 'M06',
    minggu: 13,
    tahun: 2026
  },
  jumlah_NOT_OK: 1
}

```

Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama. Pertama, `\$lookup` dan `\$unwind` digunakan untuk menggabungkan data inspeksi dengan data target kualitas, kemudian `\$addFields` menghitung persen_cacat sekaligus menentukan status kelulusan. Selanjutnya, `\$match` digunakan untuk menyaring data dengan status Not OK. Setelah itu, `\$group` berfungsi untuk menghitung jumlah kejadian Not OK berdasarkan kombinasi mesin dan minggu produksi. Terakhir, `\$sort` digunakan untuk mengurutkan hasil berdasarkan tahun, mesin, dan minggu.

4.8. Pipeline 4, Top 3 Mesin dengan Jumlah NOT OK Terbanyak (dalam satu bulan tertentu):

```

db.inspeksi.aggregate([
  { $match: {
    tanggal: { $gte: ISODate("2026-05-01"), $lt: ISODate("2026-06-01") }
  }},
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] }
  }},
  { $sort: {
    tahun: -1,
    mesin: 1,
    minggu: 1
  }},
  { $limit: 3 }
])

```

```

    }},
    { $addFields: {
      status: { $cond: { if: { $gt: ["$persen_cacat", "$target_info.target_maks_cacat"] }, then: "NOT OK", else: "OK" } }
    }},
    { $match: { status: "NOT OK" } },
    { $group: {
      _id: "$mesin",
      total_NOT_OK: { $sum: 1 }
    }},
    { $sort: { total_NOT_OK: -1 } },
    { $limit: 3 }
  ]
}

```

Screenshoot Output:

```

< {
  _id: 'M08',
  total_NOT_OK: 22
}
{
  _id: 'M06',
  total_NOT_OK: 17
}
{
  _id: 'M03',
  total_NOT_OK: 16
}

```

Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama. Pertama, `\$match` digunakan untuk memfilter data hanya pada bulan Mei 2026, kemudian `\$lookup` dan `\$unwind` menggabungkan data inspeksi dengan target kualitas untuk menghitung persen_cacat. Selanjutnya, `\$addFields` digunakan untuk menentukan status batch, lalu `\$match` kembali digunakan untuk menyaring data dengan status Not OK. Setelah itu, `\$group` menghitung total kejadian Not OK per mesin dan diurutkan menggunakan `\$sort` dari jumlah terbesar. Terakhir, `\$limit` membatasi hasil dengan menampilkan 3 mesin dengan frekuensi kegagalan tertinggi.

4.9. Pipeline 5 (Bonus/Opsional), Ringkasan Bulanan dengan \$facet:

```

db.inspeksi.aggregate([
  { $match: { tanggal: { $gte: ISODate("2026-06-01"), $lt: ISODate("2026-07-01") } } },
  { $lookup: {
    from: "target_kualitas",
    localField: "batch_id",
    foreignField: "batch_id",
    as: "target_info"
  }},
  { $unwind: "$target_info" },
  { $addFields: {
    persen_cacat: { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] },
  } }
])

```

```

    status: { $cond: { if: { $gt: [ { $multiply: [ { $divide: ["$cacat_ditemukan", "$jumlah_diperiksa"] }, 100 ] },
"$target_info.target_maks_cacat" ] }, then: "NOT OK", else: "OK" } }
  }},
  { $facet: {
    "total_batch": [
      { $count: "count" }
    ],
    "rata_cacat": [
      { $group: { _id: null, avg_cacat: { $avg: "$persen_cacat" } } }
    ],
    "mesin_terburuk": [
      { $match: { status: "NOT OK" } },
      { $group: { _id: "$mesin", jumlah: { $sum: 1 } } },
      { $sort: { jumlah: -1 } },
      { $limit: 3 }
    ]
  }
}
})

```

Screenshoot Output:

```

< {
  total_batch: [
    {
      count: 481
    }
  ],
  rata_cacat: [
    {
      _id: null,
      avg_cacat: 4.67685864004988
    }
  ],
  mesin_terburuk: [
    {
      _id: 'M09',
      jumlah: 16
    },
    {
      _id: 'M05',
      jumlah: 15
    },
    {
      _id: 'M04',
      jumlah: 15
    }
  ]
}

```

Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama untuk membangun ringkasan kualitas. Pertama, `\$match` digunakan untuk memfilter data pada bulan Juni 2026, kemudian `\$lookup` dan `\$unwind` mengambil data target guna menghitung persen_cacat sekaligus menentukan status kelulusan. Terakhir, `\$facet` digunakan untuk menghasilkan beberapa ringkasan sekaligus, yaitu total volume pemeriksaan, rata-rata tingkat kerusakan, serta tiga mesin dengan jumlah status Not OK terbanyak.

BAB V

LATIHAN MANDIRI

5.1. Gunakan database latihan5, koleksi data_sensor dengan 5000 dokumen (field: sensor_id, nilai, timestamp, category):

```
from pymongo import MongoClient
from datetime import datetime, timedelta
import random

NIM = "225443053"
client = MongoClient('mongodb://localhost:27017')

db = client[f'latihan5_{225443053}']

db.data_sensor.drop()

sensor_list = [f"S{i:03d}" for i in range(1, 21)] # S001 sampai S020
kategori_list = ["suhu", "tekanan", "kelembaban", "getaran", "arus"]

docs = []
start_date = datetime(2026, 4, 1, 0, 0)

for i in range(5000):
    kategori = random.choice(kategori_list)

    if kategori == "suhu":
        nilai = round(random.uniform(25.0, 95.0), 2)
    elif kategori == "kelembaban":
        nilai = round(random.uniform(40.0, 99.0), 2)
    else:
        nilai = round(random.uniform(10.0, 150.0), 2)

    timestamp = start_date + timedelta(minutes=random.randint(0, 60*24*30))

    doc = {
        "sensor_id": random.choice(sensor_list),
        "nilai": nilai,
        "timestamp": timestamp,
        "category": kategori
    }

    docs.append(doc)
```

```

    if len(docs) >= 500:
        db.data_sensor.insert_many(docs)
        docs.clear()

if docs:
    db.data_sensor.insert_many(docs)

print(f"Database: latihan5_{NIM}")
print("Data sensor selesai dimasukkan. Total:",
db.data_sensor.count_documents({}), "dokumen.")

```

Screenshoot Output:

```

Database: latihan5_225443053
Data sensor selesai dimasukkan. Total: 5000 dokumen.

```

5.2. Menampilkan rata-rata nilai per sensor_id pada 7 hari terakhir:

```

db.data_sensor.aggregate([
// Stage 1: Filter data 7 hari terakhir dari sekarang (Mei 2026)
{
  $match: {
    timestamp: { $gte: new Date("2026-04-27T00:00:00Z") }
  }
},
// Stage 2: Kelompokkan berdasarkan ID dan hitung rata-rata
{
  $group: {
    _id: "$sensor_id",
    rata_rata_nilai: { $avg: "$nilai" }
  }
},
// Stage 3: Urutkan
{ $sort: { _id: 1 } }
])

```

Screenshoot Output:

```

>_MONGOSH
< {
  _id: 'S001',
  rata_rata_nilai: 72.97175
}
{
  _id: 'S002',
  rata_rata_nilai: 69.06935483870969
}
{
  _id: 'S003',
  rata_rata_nilai: 65.865
}
{
  _id: 'S004',
  rata_rata_nilai: 73.95724137931035
}
{
  _id: 'S005',
  rata_rata_nilai: 70.62384615384616
}
{
  _id: 'S006',
  rata_rata_nilai: 68.02108108108109
}
{
  _id: 'S007',
  rata_rata_nilai: 77.66108108108108
}
{
  _id: 'S008',
  rata_rata_nilai: 74.53
}
{
  _id: 'S009',
  rata_rata_nilai: 66.34206896551724
}
{
  _id: 'S010',

```

Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama. Pertama, `\$match` digunakan untuk memfilter data dalam rentang 7 hari terakhir. Selanjutnya, `\$group` berfungsi untuk mengelompokkan data berdasarkan id sensor sekaligus menghitung nilai rata-rata. Terakhir, `\$sort` digunakan untuk mengurutkan hasil berdasarkan id sensor.

5.3. Menampilkan sensor yang memiliki nilai di atas 90 minimal 3 kali (gunakan \$group dengan \$push atau \$sum kondisional).

```

db.data_sensor.aggregate([
  // Stage 1: Filter hanya dokumen yang nilainya di atas 90
  {
    $match: {
      nilai: { $gt: 90 }
    }
  },

```

```
// Stage 2: Hitung berapa kali tiap sensor muncul setelah difilter
{
  $group: {
    _id: "$sensor_id",
    frekuensi_tinggi: { $sum: 1 }
  }
},
// Stage 3: Filter sensor yang muncul minimal 3 kali
{
  $match: {
    frekuensi_tinggi: { $gte: 3 }
  }
}
}
```

Screenshoot Output:

```
>_MONGOSH
< {
  _id: 'S016',
  frekuensi_tinggi: 86
}
{
  _id: 'S003',
  frekuensi_tinggi: 69
}
{
  _id: 'S012',
  frekuensi_tinggi: 71
}
{
  _id: 'S009',
  frekuensi_tinggi: 73
}
{
  _id: 'S006',
  frekuensi_tinggi: 80
}
{
  _id: 'S017',
  frekuensi_tinggi: 75
}
{
  _id: 'S013',
  frekuensi_tinggi: 72
}
{
  _id: 'S014',
  frekuensi_tinggi: 80
}
{
  _id: 'S018',
  frekuensi_tinggi: 78
}
{
  _id: 'S007',
```


Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama. Pertama, `\$match` digunakan untuk menyaring dokumen dengan nilai di atas 90. Selanjutnya, `\$group` berfungsi untuk menghitung frekuensi kemunculan menggunakan `\$sum: 1`, sehingga setiap data yang memenuhi kondisi akan dihitung. Terakhir, `\$match` kembali digunakan untuk menyeleksi sensor yang memiliki frekuensi tinggi minimal 3 kali.

5.4. Menambahkan field jam dan menghitung nilai rata-rata per jam per sensor.

```
db.data_sensor.aggregate([
  {
    $addFields: {
      jam: { $hour: "$timestamp" }
    }
  },
  {
    $group: {
      _id: {
        sensor_id: "$sensor_id",
        jam: "$jam"
      },
      rata_rata: { $avg: "$nilai" }
    }
  },
  {
    $sort: {
      "_id.sensor_id": 1,
      "_id.jam": 1
    }
  }
])
```

Screenshoot Output:

```

>_MONGOSH
< {
  _id: {
    sensor_id: 'S001',
    jam: 0
  },
  rata_rata: 69.835
}
{
  _id: {
    sensor_id: 'S001',
    jam: 1
  },
  rata_rata: 61.32636363636364
}
{
  _id: {
    sensor_id: 'S001',
    jam: 2
  },
  rata_rata: 71.30076923076923
}
{
  _id: {
    sensor_id: 'S001',
    jam: 3
  },
  rata_rata: 85.73555555555555
}
{
  _id: {
    sensor_id: 'S001',
    jam: 4
  },
  rata_rata: 84.47200000000001
}
{
  _id: {
    sensor_id: 'S001',
    jam: 5
  },
  rata_rata: 84.47200000000001
}

```

Penjelasan: Pada bagian ini, pipeline dibagi ke dalam beberapa stage utama. Pertama, \$addField digunakan untuk menambah informasi waktu dari timestamp menjadi field jam. Kedua, \$group digunakan untuk mengelompokkan data id sensor, sekaligus menghitung rata_rata_jam. Terakhir, \$sort menyusun hasil berdasarkan id sensor dan waktu.

5.5. Buat koleksi sensor_info lalu join untuk menampilkan nama sensor dalam hasil

```

var info = [];
for (var i = 1; i <= 20; i++) {
  var id = "S" + i.toString().padStart(3, '0');
  info.push({
    sensor_id: id,
    nama_sensor: "Device-" + id,
    lokasi: "Lantai " + (Math.floor(Math.random() * 3) + 1),
    status: "Aktif"
  });
}
db.sensor_info.insertMany(info);

db.data_sensor.aggregate([

```

```
{
  $group: {
    _id: "$sensor_id",
    rata_rata: { $avg: "$nilai" }
  },
  {
    $lookup: {
      from: "sensor_info",
      localField: "_id",
      foreignField: "sensor_id",
      as: "detail_sensor"
    }
  },
  { $unwind: "$detail_sensor" },
  {
    $project: {
      _id: 1,
      rata_rata: 1,
      lokasi: "$detail_sensor.lokasi",
    }
  }
}
```

Screenshoot Output:

```

>_MONGOSH
< {
  _id: 'S003',
  rata_rata: 73.97314814814816,
  lokasi: 'Lantai 1'
}
{
  _id: 'S003',
  rata_rata: 73.97314814814816,
  lokasi: 'Lantai 3'
}
{
  _id: 'S007',
  rata_rata: 76.75075098814229,
  lokasi: 'Lantai 1'
}
{
  _id: 'S007',
  rata_rata: 76.75075098814229,
  lokasi: 'Lantai 2'
}
{
  _id: 'S004',
  rata_rata: 76.27362139917696,
  lokasi: 'Lantai 1'
}
{
  _id: 'S004',
  rata_rata: 76.27362139917696,
  lokasi: 'Lantai 1'
}
{
  _id: 'S010',
  rata_rata: 71.2010843373494,
  lokasi: 'Lantai 2'
}
{
  _id: 'S010',
  rata_rata: 71.2010843373494,

```

Penjelasan: Pada bagian ini, pipeline terdiri dari beberapa tahap utama. Pertama, `\$group` digunakan untuk menghitung nilai rata-rata dari data sensor. Selanjutnya, `\$lookup` dan `\$unwind` berfungsi untuk menggabungkan hasil tersebut dengan koleksi sensor_info. Terakhir, `\$project` digunakan untuk menampilkan informasi yang diperlukan, yaitu id sensor, nilai rata-rata, dan lokasi.

BAB VI

ANALISIS DAN KESIMPULAN

6.1. Analisis

Berdasarkan hasil eksekusi lima pipeline agregasi, dapat disimpulkan bahwa kualitas produksi di pabrik masih belum stabil. Terdapat beberapa mesin yang menghasilkan persentase cacat cukup tinggi dan bahkan melampaui batas toleransi, sehingga menunjukkan bahwa proses produksi belum sepenuhnya optimal. Selain itu, perbandingan antara hasil aktual dan target kualitas juga memperlihatkan bahwa masih banyak batch yang belum memenuhi standar yang ditetapkan.

Analisis lebih lanjut menunjukkan adanya fluktuasi jumlah batch NOT OK dari waktu ke waktu, yang mengindikasikan kemungkinan gangguan operasional atau kurang optimalnya pemeliharaan mesin. Beberapa mesin juga secara konsisten muncul sebagai penyumbang kegagalan tertinggi dalam periode tertentu, sehingga perlu menjadi prioritas untuk diperbaiki.

Secara keseluruhan, meskipun rata-rata tingkat cacat masih dalam batas wajar, adanya pola masalah yang berulang pada mesin tertentu menunjukkan perlunya evaluasi dan perbaikan lebih lanjut agar kualitas produksi dapat lebih terjaga dan stabil.

6.2. Kesimpulan

Melalui penerapan Aggregation Framework di MongoDB, mahasiswa berhasil menyusun pipeline multi-stage yang mampu mengolah data inspeksi kualitas secara efisien langsung di database. Lima pipeline yang dibuat memanfaatkan berbagai operator penting seperti `$addFields` untuk perhitungan, `$lookup` untuk menggabungkan data antar koleksi, `$match` untuk penyaringan, `$group` untuk pengelompokan, `$sort` untuk pengurutan, serta `$facet` untuk menghasilkan laporan secara bersamaan.

Selain itu, penggunaan teknik optimasi seperti menempatkan `$match` di awal pipeline dan membuat indeks pada field `batch_id`, mesin, dan tanggal terbukti membuat proses eksekusi menjadi lebih cepat. Dari hasil analisis, terlihat bahwa mesin M03, M04, dan M09 merupakan mesin yang paling bermasalah, dengan jenis cacat yang sering muncul seperti gores, retak, dan bengkok. Oleh karena itu, disarankan untuk segera melakukan pemeriksaan menyeluruh dan perawatan rutin pada mesin-mesin tersebut agar kualitas produksi dapat ditingkatkan.